

The Socket Exchange

COL334 — Assignment 2 — Experiment Report

Rohit Meena (2024CS10030) • Subarno Saha (2024CS50431)

FreeBSD 14.4-RELEASE • C++17 • September 10, 2026

All experiments were run on FreeBSD 14.4-RELEASE. The Exchange Server is written in C++17 against the POSIX socket API with `kqueue()`; no third-party networking library is used. Every measurement below was taken on the running system with the FreeBSD tools named in each section.

Contents

1	Implementation Decisions	2
1.1	Concurrency / I/O approach	2
1.2	Why this approach	2
1.3	Other significant design decisions	2
2	Experiments	4
2.1	Experiment 1 — Listening and Connected Sockets	4
2.2	Experiment 2 — Observing TCP Connection States	5
2.3	Experiment 3 — TCP as a Byte Stream	9
2.4	Experiment 4 — One Client Should Not Stall the Others	11
2.5	Experiment 5 — Multiple Clients and I/O Multiplexing	12
2.6	Experiment 6 — FIN vs. RST	14
2.7	Experiment 7 — Backpressure and the Slow Receiver	16
2.8	Experiment 8 — Unexpected Client Disconnection	19
3	Bonus — Connection Scalability and I/O Design	21
3.1	Client-generation program	21
3.2	Resource-measurement table	21
3.3	Q1 — Able to maintain all requested connections?	22
3.4	Q2 — First significant bottleneck, with supporting data	22
3.5	Q3 — How the concurrency/I/O design contributes	23
3.6	Q4 — Would changing the I/O mechanism help?	23
3.7	Q5 — Quantified trade-off after a change	23
3.8	Reproducing the tuning	24
3.9	Bonus screenshots	24

1 Implementation Decisions

1.1 Concurrency / I/O approach

The Exchange Server is a **single-threaded event loop built on `kqueue()`**. There is one process and one thread for the entire server. The listening socket and every accepted connection are registered on a single `kqueue` with `EVFILT_READ`; `kevent()` blocks until the kernel reports descriptors ready, and the loop services only those.

This is verifiable from outside the process. `procstat -f` shows the server holding the listening socket on fd 3, **the `kqueue` itself on fd 4** (descriptor type `k`), and one socket fd per connection thereafter (Figures 2.1 and 2.5). `procstat -k` shows exactly one thread, asleep inside `sys_kevent`:

```
mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan
kqueue_kevent kern_kevent_fp kern_kevent_generic sys_kevent
amd64_syscall fast_syscall_common
```

1.2 Why this approach

Cost scales with activity, not with connection count. `kevent()` returns only descriptors the kernel has already marked ready, so a loop iteration costs $O(\text{ready})$ rather than $O(\text{open})$. Experiment 5 shows this directly: with five connections open and only three carrying data, the server called `recv()` on exactly those three and never touched the other two.

One unresponsive client cannot stall the others. Because the loop blocks in `kevent()` rather than in `recv()` on a particular socket, a client that connects and then sends a partial message is simply never reported ready. Experiment 4 measured a second client being served in 0.001s while the first held an unterminated message, with `procstat -k` confirming the server was parked in `sys_kevent` and not in `soreceive`.

Per-connection cost is tiny, which is what makes the bonus possible. Measured across seven levels, server resident memory grows by ≈ 2012 KB per 10,000 idle connections — about **201 bytes per connection** — on a 3.4 MB baseline. Holding 70,000 idle connections cost 17.1 MB of RSS and 0.1% CPU.

The alternatives are worse. A thread per connection would need 70,000 stacks; even at a conservative 512 KB each that is ≈ 34 GB against 4 GB of RAM, roughly four orders of magnitude more than the measured 201 bytes. `select()` is bounded by `FD_SETSIZE` (1024 by default) and so cannot reach these numbers at all. `poll()` has no such cap but requires passing and scanning the whole descriptor array on every call — $O(n)$ in the number of *open* descriptors, i.e. 70,000 entries examined to find the few that are ready.

1.3 Other significant design decisions

All sockets are non-blocking. `set_nonblocking()` is applied to every descriptor returned by `accept()`. This is mandatory rather than optional: in a readiness-based loop a single blocking `recv()` or `send()` would stall every other client, which is exactly the failure Experiment 4 tests for.

Message framing is explicit, because TCP provides none. Each connection owns a `LineBuffer` that accumulates bytes from `recv()` and yields complete `\n`-terminated lines. One `recv()` is never assumed to equal one message. Experiment 3 demonstrates why: a single 24-byte message sent in four writes arrived as four separate `recv()` calls of 6, 10, 7 and 1 bytes, and the server produced no command until the final newline arrived. On the output side the `\n` is appended in exactly one place (`enqueue()`), so a line can never be double-framed.

Backpressure is handled per connection, so a slow reader is contained. `enqueue()` first attempts `send()` directly. If the peer's receive window is full, `send()` returns `EAGAIN` — treated as backpressure, not as an error — and the unsent remainder is appended to that connection's own `outbuf` queue, with `EVFILT_WRITE` registered for that socket only. When the socket becomes writable the queue is drained and the filter is removed again. Consequently a client that stops reading backs up on its own socket alone. Experiment 7 confirms this: with one market-data client deliberately not reading, its receive queue grew to 85,000 bytes while the normal client stayed at 17 bytes and both traders completed all 5,000 trades.

The accept loop drains fully. A single `EVFILT_READ` on the listening socket can represent several queued connections, so `accept()` is called in a loop until it returns `EAGAIN`. Without this the server would fall behind under burst load — which matters at the connection rates used in the bonus.

Connection teardown is idempotent. `close_session()` returns immediately if the session is already gone, and `EV_EOF` is acted on only after `on_readable()` has run, so a descriptor is never closed twice. `recv()` returning 0 (FIN) and `recv()` failing with `ECONNRESET` (RST) are logged distinctly, which is what made Experiments 6 and 8 measurable from the application side.

Roles are inferred from the first command. A connection is `UNKNOWN` until it issues a command; `LOGIN`/`BUY`/`SELL`/`CANCEL` make it a Trader, `SUBSCRIBE`/`UNSUBSCRIBE` a Market-Data client. A command outside the connection's role is answered with `ERROR`, which is what makes a Market-Data connection genuinely read-only.

Resting orders outlive their owner's connection (handout 2.6). `close_session()` deliberately does *not* remove the departing client's orders; they remain in the book and can still match. Subscribed Market-Data clients still receive the resulting `TRADE` messages, while the departed trader receives no `BOUGHT`/`SOLD`.

A generation counter guards against descriptor reuse. Every session carries a monotonically increasing `session_seq`, recorded on each order. Before delivering a fill the server checks that the descriptor's *current* session sequence still matches the one that submitted the order. Without this, a fill belonging to a departed trader could be delivered to whichever new client happened to inherit that file descriptor number — a real hazard given how aggressively descriptors are recycled.

2 Experiments

2.1 Experiment 1 — Listening and Connected Sockets

Question. After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

Answer. The server holds **two distinct TCP sockets on two distinct file descriptors**. Fd 3 is the *listening* socket: bound to the local half-association 127.0.0.1:5000 only, with a wildcard foreign address `*,*`, in state `LISTEN`. It carries no peer and no data and exists solely to be `accept()`ed from. Fd 5 is the *connected* socket returned by that `accept()`: fully specified by the four-tuple 127.0.0.1:5000 $\leftrightarrow$ 127.0.0.1:51080, in state `ESTABLISHED`, and the only one of the two over which bytes flow. Both share local port 5000; it is the *remote* endpoint that distinguishes them, which is precisely why one listening port can fan out to many simultaneous connections.

Approach. Ran the harness so the server was listening with exactly one client attached, then inspected the same instant from two angles: the system-wide socket table (which addresses and states exist) and the server process's own descriptor table (which fds those sockets occupy inside the process).

Commands and tools.

```
sockstat -4l | grep 5000          # baseline: port free before start
sockstat -4 | grep -E 'USER|exchange|5000' # server sockets by process and fd
netstat -an -p tcp | grep -E 'Proto|5000'  # addresses and TCP states
procstat -f $(pgrep -x exchange_server)    # the server's fd table
```

Evidence. `sockstat` shows fd 3 $\rightarrow$ `*,*` and fd 5 $\rightarrow$ 127.0.0.1:51080. `netstat` shows one `LISTEN` row and *two* `ESTABLISHED` rows (both directions of the loopback pair). `procstat -f` additionally shows fd 4 of type `k` — the `kqueue`. `Recv-Q` and `Send-Q` are 0 on every row: an idle connection buffers nothing.

```
vzux@freebsd:~ $ sockstat -4l | grep 5000 ; netstat -an -p tcp | grep 5000 ; echo "(no rows above = port free)"
(no rows above = port free)
vzux@freebsd:~ $ { echo "=== sockstat -4 ==="; sockstat -4 | grep -E 'USER|exchange|5000'; \
> echo; echo "=== netstat -an -p tcp ==="; netstat -an -p tcp | grep -E 'Proto|5000'; } \
> | tee -/evidence/exp1/exp1_sockets.txt
=== sockstat -4 ===
USER      COMMAND   PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
vzux      exchange_s 1133 3   tcp4   127.0.0.1:5000  *,*
vzux      exchange_s 1133 5   tcp4   127.0.0.1:5000  127.0.0.1:51080
vzux      python3.12 1131 3   tcp4   127.0.0.1:51080  127.0.0.1:5000

=== netstat -an -p tcp ===
Proto Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4      0      0 127.0.0.1.5000          127.0.0.1.51080        ESTABLISHED
tcp4      0      0 127.0.0.1.51080        127.0.0.1.5000        ESTABLISHED
tcp4      0      0 127.0.0.1.5000          *,*                     LISTEN
vzux@freebsd:~ $
```

Figure 1: Experiment 1 — baseline showing the port free, then `sockstat` and `netstat` with the `LISTEN` socket (`*,*`) and both `ESTABLISHED` rows of the loopback pair.

```

vzux@freebsd:~ $ { echo "=== sockstat -4 ==="; sockstat -4 | grep -E 'USER|exchange|5000'; \
> echo; echo "=== netstat -an -p tcp ==="; netstat -an -p tcp | grep -E 'Proto|5000'; } \
> | tee ~/evidence/exp1/exp1_sockets.txt
=== sockstat -4 ===
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
vzux      exchange_s 1133 3   tcp4   127.0.0.1:5000  *:
vzux      exchange_s 1133 5   tcp4   127.0.0.1:5000  127.0.0.1:51080
vzux      python3.12 1131 3   tcp4   127.0.0.1:51080  127.0.0.1:5000

=== netstat -an -p tcp ===
Proto Recv-Q Send-Q Local Address      Foreign Address    (state)
tcp4   0      0 127.0.0.1:5000     127.0.0.1:51080   ESTABLISHED
tcp4   0      0 127.0.0.1:51080   127.0.0.1:5000   ESTABLISHED
tcp4   0      0 127.0.0.1:5000     *:                LISTEN

vzux@freebsd:~ $ procstat -f $(pgrep -x exchange_server) | tee ~/evidence/exp1/exp1_fds.txt
PID COMM  FD T V FLAGS  REF  OFFSET PRO NAME
1133 exchange_server text v r r----- - - - /home/vzux/A2_ROLL1_ROLL2/bin/exchange_server
1133 exchange_server cwd v d r----- - - - /home/vzux/A2_ROLL1_ROLL2
1133 exchange_server root v d r----- - - - /
1133 exchange_server 0 v c rw----- 7 1945 - /dev/pts/0
1133 exchange_server 1 p - rw----- 4 0 - -
1133 exchange_server 2 p - rw----- 4 0 - -
1133 exchange_server 3 s - rw----- 2 0 TCP 0 0 127.0.0.1:5000 *:0
1133 exchange_server 4 k - rw----- 2 0 - -
1133 exchange_server 5 s - rw----- 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:51080
vzux@freebsd:~ $

```

Figure 2: Experiment 1 — `procstat -f`: fd 3 listening socket, fd 4 the **kqueue** (type k), fd 5 the connected socket. One process holds all three.

2.2 Experiment 2 — Observing TCP Connection States

Question. How does the TCP state of the client–server connection change during its lifetime, and what events cause the observed state changes?

Answer.

Establishment. `connect()` sends SYN; the server replies SYN+ACK; the client ACKs. Both endpoints reach ESTABLISHED. On loopback this took $20\mu\text{s}$ (SYN at 06:14:20.569648, final ACK at .569668), so SYN_SENT and SYN_RCVD never appear in a 1-second `netstat` sample — they exist only in the packet capture.

Active phase. Both rows remain ESTABLISHED with Recv-Q = Send-Q = 0 for the full 10-second idle window.

Termination. The client closes first, sending FIN at 06:14:30.658923. The server ACKs it and its `recv()` returns 0 — logged as [5] `recv() -> 0 : peer closed (FIN)` — whereupon the event loop closes the socket, emitting the server’s own FIN at .659202, which the client ACKs.

The critical timing result. The gap between the two FINs is **279 μs** (Pass A) and **143 μs** (Pass B). That interval *is* the server’s CLOSE_WAIT and the client’s FIN_WAIT_2 state. Both are real but last only microseconds, because the event loop reacts to `recv()==0` immediately. Their absence from `netstat` is a sampling artefact, not an absence of the states.

TIME_WAIT and the loopback special case. This guest has `net.inet.tcp.nolocaltimewait = 1`, so FreeBSD deliberately skips TIME_WAIT when both endpoints are local. Hence in Pass A the two rows simply vanish at 06:14:31, leaving only LISTEN. Setting the `sysctl` to 0 (Pass B) restores standard behaviour: the closing side enters TIME_WAIT at 06:36:41 and holds it through 06:37:40, disappearing at 06:37:41 — **exactly 60 s = $2 \times \text{MSL}$** , with `net.inet.tcp.msl = 30000 ms`.

TIME_WAIT is kernel state, not process state. The harness stopped the server 15 s after the close: the LISTEN row is present at 06:36:56 and gone at 06:36:57, yet TIME_WAIT survives a further **44 seconds**, to 06:37:40. The four-tuple is held by the kernel long after the owning process has exited — which is the purpose of TIME_WAIT: preventing a delayed duplicate segment from being delivered to a new connection reusing that four-tuple. The contrast is direct: in Pass A the LISTEN row dies *with* the process and nothing outlives it.

Incidental observation. The harness’s readiness probe reaches port 5000 before the server has bound,

at 06:14:20.462364, and its SYN is answered 32 μ s later by RST — connection refused. A SYN to a port with no listener produces RST, never FIN. This is reused in Experiment 6.

Approach. Ran the experiment twice with three terminals running concurrently: a packet capture started first, a 1-second netstat sampler second, the harness last. Pass A used the guest default; Pass B set `nolocaltimewait=0` to expose `TIME_WAIT`, and the sampler was deliberately left running past the end of the harness so the `TIME_WAIT` lifetime could be measured against the disappearance of the `LISTEN` row.

Commands and tools.

```
su root -c 'tcpdump -l -i lo0 -n -tttt "tcp port 5000" | tee exp2A_tcpdump.txt
while true; do date +%T; netstat -an -p tcp | grep 5000; echo; sleep 1; done \
  | tee exp2A_states.txt
python3 -u experiment.py 2 2>&1 | tee exp2A_harness.txt
su root -c 'sysctl net.inet.tcp.nolocaltimewait=0'    # Pass B; restored to 1 after
sysctl net.inet.tcp.msl                             # = 30000 ms
```

The `-l` flag is essential: without it `tcpdump` block-buffers into the pipe and prints nothing until it exits.

Time	Protocol	Local Address	Foreign Address	State
06:14:20	tcp4	0.0.0.0 127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.5000	*,*	LISTEN
06:14:21	tcp4	0.0.0.0 127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.5000	*,*	LISTEN
06:14:22	tcp4	0.0.0.0 127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.5000	*,*	LISTEN
06:14:23	tcp4	0.0.0.0 127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.5000	*,*	LISTEN
06:14:24	tcp4	0.0.0.0 127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0.0.0.0 127.0.0.1.5000	*,*	LISTEN

Figure 3: Experiment 2, Pass A — steady ESTABLISHED (both directions) plus LISTEN, 06:14:20–24.

06:14:28	tcp4	0	0	127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0	0	127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
06:14:30	tcp4	0	0	127.0.0.1.5000	127.0.0.1.47370	ESTABLISHED
	tcp4	0	0	127.0.0.1.47370	127.0.0.1.5000	ESTABLISHED
	tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
06:14:31	tcp4	0	0	127.0.0.1.5000	*,*	LISTEN

Figure 4: Experiment 2, Pass A — the transition. ESTABLISHED at 06:14:28 and :30; at 06:14:31 both rows have vanished with **no** `TIME_WAIT`, because `nolocaltimewait=1` skips it on loopback.

```

vzux@freebsd:~$ su root -c 'tcpdump -i lo0 -n -tttt "tcp port 5000" | tee ~/evidence/exp2/exp2A_tcpdump.txt
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
^C13 packets captured
13 packets received by filter
0 packets dropped by kernel
2026-09-09 06:14:20.462364 IP 127.0.0.1.30151 > 127.0.0.1.5000: Flags [S], seq 3171401802, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 778335757 ecr 0], length 0
2026-09-09 06:14:20.462396 IP 127.0.0.1.5000 > 127.0.0.1.30151: Flags [R.], seq 0, ack 3171401803, win 0, length 0
2026-09-09 06:14:20.568887 IP 127.0.0.1.25180 > 127.0.0.1.5000: Flags [S], seq 1959940144, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 973421686 ecr 0], length 0
2026-09-09 06:14:20.569122 IP 127.0.0.1.5000 > 127.0.0.1.25180: Flags [S.], seq 578008021, ack 1959940145, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 557523682 ecr 973421686], length 0
2026-09-09 06:14:20.569147 IP 127.0.0.1.25180 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 973421686 ecr 557523682], length 0
2026-09-09 06:14:20.569322 IP 127.0.0.1.25180 > 127.0.0.1.5000: Flags [R.], seq 1, ack 1, win 0, options [nop,nop,TS val 973421686 ecr 557523682], length 0
2026-09-09 06:14:20.569648 IP 127.0.0.1.47370 > 127.0.0.1.5000: Flags [S], seq 609596704, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2390145657 ecr 0], length 0
2026-09-09 06:14:20.569655 IP 127.0.0.1.5000 > 127.0.0.1.47370: Flags [S.], seq 2610488172, ack 609596705, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1708953775 ecr 2390145657], length 0
2026-09-09 06:14:20.569668 IP 127.0.0.1.47370 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 2390145657 ecr 1708953775], length 0
2026-09-09 06:14:30.658923 IP 127.0.0.1.47370 > 127.0.0.1.5000: Flags [F.], seq 1, ack 1, win 1277, options [nop,nop,TS val 2390155748 ecr 1708953775], length 0
2026-09-09 06:14:30.658992 IP 127.0.0.1.5000 > 127.0.0.1.47370: Flags [.], ack 2, win 1277, options [nop,nop,TS val 1708963866 ecr 2390155748], length 0
2026-09-09 06:14:30.659202 IP 127.0.0.1.5000 > 127.0.0.1.47370: Flags [F.], seq 1, ack 2, win 1277, options [nop,nop,TS val 1708963866 ecr 2390155748], length 0
2026-09-09 06:14:30.659211 IP 127.0.0.1.47370 > 127.0.0.1.5000: Flags [.], ack 2, win 1277, options [nop,nop,TS val 2390155748 ecr 1708963866], length 0
vzux@freebsd:~$

```

Figure 5: Experiment 2, Pass A — the complete 13-packet capture: the connection-refused RST, the three-way handshake, and the four-way FIN close.

```

vzux@mycomputer: ~
* FreeBSD experiments with data... x vzux@mycomputer: ~ x vzux@mycomputer: ~ x vzux@mycomputer: ~ x vzux@mycomputer: ~
06:14:37      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:38      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:39      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:40      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:41      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:42      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:43      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:44      0      0 127.0.0.1.5000      *.*          LISTEN
06:14:45
06:14:47
06:14:48
06:14:49
06:14:50
06:14:51
06:14:52
06:14:53
06:14:54

```

Figure 6: Experiment 2, Pass A — LISTEN persists to 06:14:44 and disappears at :45 when the server process exits. Nothing outlives the process.

```

^C
vzux@freebsd:~$ su root -c 'sysctl net.inet.tcp.nolocaltimewait=0'
net.inet.tcp.nolocaltimewait: 1 -> 0
vzux@freebsd:~$

```

Figure 7: Experiment 2, Pass B — net.inet.tcp.nolocaltimewait: 1 -> 0.

06:36:56				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
tcp4	0	0 127.0.0.1.5000	*.*	LISTEN
06:36:57				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:36:58				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:36:59				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:00				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:01				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:02				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:03				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:04				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:05				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:06				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:07				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:08				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:09				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT

Figure 8: Experiment 2, Pass B — the key result. At 06:36:56 both TIME_WAIT and LISTEN are present; at 06:36:57 LISTEN is gone (the server process has exited) yet TIME_WAIT continues.

06:37:33				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:34				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:35				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:36				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:37				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:38				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:39				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:40				
tcp4	0	0 127.0.0.1.64184	127.0.0.1.5000	TIME_WAIT
06:37:41				
06:37:43				
06:37:44				
06:37:45				
06:37:46				
06:37:47				
06:37:48				
06:37:49				
06:37:50				
06:37:51				

Figure 9: Experiment 2, Pass B — last TIME_WAIT sample at 06:37:40, gone at 06:37:41: exactly 60s = 2×MSL after the close at 06:36:41.

```
vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && python3 -u experiment.py 2 2>&1 | tee ~/evidence/exp2/exp2B_harness.txt
=== Experiment 2: Observing TCP Connection States ===

Started Exchange Server (PID 3924).
exchange_server listening on 127.0.0.1:5000 (kqueue fd=4, listen fd=3)
[+] accepted fd=5 from 127.0.0.1:54348 (1 client(s) connected)
[5] recv() -> ECONNRESET : peer reset (RST)
Exchange Server is listening.

[-] fd=5 closed (0 client(s) still connected)
[+] accepted fd=5 from 127.0.0.1:64184 (1 client(s) connected)
Experiment client: connected from 127.0.0.1:64184

Phase 1: the client is connected and idle.
Investigate the connection now.

Phase 2: closing the client connection.
Investigate the connection again.

The Exchange Server will remain running for 15 seconds.[5] recv() -> 0 : peer closed (FIN)

[-] fd=5 closed (0 client(s) still connected)

Experiment finished.
Stopping Exchange Server...
vzux@freebsd:~/A2_ROLL1_ROLL2 $ pgrep -x exchange_server || echo "SERVER PROCESS IS GONE" ; netstat -an -p tcp | grep 5000
SERVER PROCESS IS GONE
vzux@freebsd:~/A2_ROLL1_ROLL2 $
```

Figure 10: Experiment 2, Pass B — server trace: accept, recv() -> 0 : peer closed (FIN), close.

2.3 Experiment 3 — TCP as a Byte Stream

Question. Does the Exchange Server receive the application-level message as one complete unit, or can it be received in multiple pieces? What does this demonstrate about TCP and application-level message boundaries?

Answer. In multiple pieces. The single message `LOGIN experiment_trader\n` (24 bytes) was written by the client as four separate `send()` calls and arrived as **four separate** `recv()` calls **returning 6, 10, 7 and 1 bytes** — never as one 24-byte unit. The server could act on none of them until the fourth delivered the terminating newline, at which point it emitted `[5] LOGIN experiment_trader`.

This demonstrates that TCP is a *byte stream*, not a message stream. It guarantees that bytes arrive in order and without loss, but preserves no record of where the sender's writes began or ended. Application-level message boundaries do not exist at the transport layer, so the application must impose them — here by buffering and splitting on `\n`. The converse also holds: had the pieces arrived close enough together they would have coalesced into one `recv()`, and several messages can equally arrive in one `recv()`. Either way the receiver cannot assume one `recv()` equals one message, which is why the `LineBuffer` accumulator is required rather than optional.

Approach. Observed the same event from both sides simultaneously: the server's own instrumentation logging the return value of every `recv()`, and a packet capture with ASCII payload decoding to confirm what actually crossed the wire. Cross-checking the two rules out the possibility that the split was an artefact of the logging rather than of TCP.

Commands and tools.

```
su root -c 'tcpdump -l -i lo0 -n -tttt -A "tcp port 5000" | tee exp3_tcpdump.txt
python3 -u experiment.py 3 2>&1 | tee exp3_harness.txt
```

`-A` prints payload as ASCII. The server logs each `recv()` return value at `exchange_server.cpp:365`; handout 6.3 explicitly permits modifying the implementation to make this observable.

Evidence. Four [P.] segments from 127.0.0.1:28206, with *contiguous* sequence numbers — TCP delivered one unbroken 24-byte stream; the segmentation reflects the sender's write pattern, not the message:

Time	Seq range	Length	Payload
15:59:02.821997	1:7	6	"LOGIN "
15:59:03.030565	7:17	10	"experiment"
15:59:03.241641	17:24	7	"_trader"
15:59:03.451747	24:25	1	"\n"
15:59:03.452274	server → client, 3		"OK"

The server's OK left 527 μ s after the newline arrived, confirming it was waiting for the delimiter and not for a message.

```
vzux@frebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && python3 -u experiment.py 3 2>&1 | tee ~/evidence/exp3/exp3_harness.txt
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 5910).
exchange_server listening on 127.0.0.1:5000 (kqueue fd=4, listen fd=3)
[+] accepted fd=5 from 127.0.0.1:58693 (1 client(s) connected)
Exchange Server is listening.

[5] recv() -> ECONNRESET : peer reset (RST)
[-] fd=5 closed (0 client(s) still connected)
Experiment client: connected from 127.0.0.1:28206[+] accepted fd=5 from 127.0.0.1:28206 (1 client(s) connected)

Sending one application-level message using several separate TCP writes...

Sent 6 bytes.
[5] recv() -> 6 byte(s)
Sent 10 bytes.
[5] recv() -> 10 byte(s)
Sent 7 bytes.
[5] recv() -> 7 byte(s)
Sent 1 bytes.
[5] recv() -> 1 byte(s)
[5] LOGIN experiment_trader

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
```

Figure 11: Experiment 3 — four `recv()` calls returning 6, 10, 7 and 1 bytes, and the single parsed LOGIN line emitted only after the fourth.

```
vzux@frebsd:~$ su root -c "tcpdump -i lo -n -s 1554 -A 'tcp port 5000' | tee ~/evidence/exp3/exp3_tcpdump.txt
tcpdump: verbose output suppressed, use -v[:...] for full protocol decode
listening on lo, link-type EN10 (Ethernet), snapshot length 16384 bytes
2026-09-09 15:59:02.713727 IP 127.0.0.1.22763 > 127.0.0.1.5000: Flags [S], seq 162088356, win 65535, options [msg 16344,nop,wscale 6,sackOK,T5 val 2828991834 ecr 8], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.713743 IP 127.0.0.1.5000 > 127.0.0.1.22763: Flags [R..], seq 0, ack 162088357, win 0, length 0
E..A..0.0.....0.....
2026-09-09 15:59:02.820517 IP 127.0.0.1.58693 > 127.0.0.1.5000: Flags [S], seq 3509051475, win 65535, options [msg 16344,nop,wscale 6,sackOK,T5 val 2171678238 ecr 8], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.820762 IP 127.0.0.1.5000 > 127.0.0.1.58693: Flags [S..], seq 2955212871, ack 3509051476, win 65535, options [msg 16344,nop,wscale 6,sackOK,T5 val 1341684840 ecr 2171678238], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.820918 IP 127.0.0.1.58693 > 127.0.0.1.5000: Flags [..], ack 1, win 1277, options [nop,nop,T5 val 2171678238 ecr 1341684840], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.821119 IP 127.0.0.1.58693 > 127.0.0.1.5000: Flags [R..], seq 1, ack 1, win 0, options [nop,nop,T5 val 2171678238 ecr 1341684840], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.821632 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [S], seq 1460781992, ack 65535, options [msg 16344,nop,wscale 6,sackOK,T5 val 84968263 ecr 8], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.821640 IP 127.0.0.1.5000 > 127.0.0.1.28206: Flags [S..], seq 2252408177, ack 1460781993, win 65535, options [msg 16344,nop,wscale 6,sackOK,T5 val 1281241387 ecr 84968263], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.821670 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [..], ack 1, win 1277, options [nop,nop,T5 val 84968263 ecr 1281241387], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.821997 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [P..], seq 1:7, ack 1, win 1277, options [nop,nop,T5 val 84968263 ecr 1281241387], length 6
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:02.857592 IP 127.0.0.1.5000 > 127.0.0.1.28206: Flags [..], ack 7, win 1277, options [nop,nop,T5 val 1281241425 ecr 84968263], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.030565 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [P..], seq 7:17, ack 1, win 1277, options [nop,nop,T5 val 84968471 ecr 1281241425], length 10
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.030565 IP 127.0.0.1.5000 > 127.0.0.1.28206: Flags [..], ack 17, win 1277, options [nop,nop,T5 val 1281241435 ecr 84968471], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.241641 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [P..], seq 17:24, ack 1, win 1277, options [nop,nop,T5 val 84968685 ecr 1281241435], length 7
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.277671 IP 127.0.0.1.5000 > 127.0.0.1.28206: Flags [..], ack 24, win 1277, options [nop,nop,T5 val 1281241843 ecr 84968685], length 0
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.451747 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [P..], seq 24:25, ack 1, win 1277, options [nop,nop,T5 val 84968895 ecr 1281241843], length 1
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.452274 IP 127.0.0.1.5000 > 127.0.0.1.28206: Flags [P..], seq 1:4, ack 25, win 1277, options [nop,nop,T5 val 1281242819 ecr 84968895], length 3
E..A..0.0.....0.....
S..0..
2026-09-09 15:59:03.487619 IP 127.0.0.1.28206 > 127.0.0.1.5000: Flags [..], ack 4, win 1277, options [nop,nop,T5 val 84969031 ecr 1281242819], length 0
E..A..0.0.....0.....
S..0..
^C
18 packets captured
18 packets received by filter
0 packets dropped by kernel
vzux@frebsd:~$
```

Figure 12: Experiment 3 — the wire view: four [P.] segments with contiguous sequence numbers and readable ASCII payloads, then the server's OK.

2.4 Experiment 4 — One Client Should Not Stall the Others

Question. When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2? Identify the server operation that determines the answer.

Answer. Yes. Client 2 connected, sent `LOGIN active_client` and received `OK` in **0.001 s** while Client 1 was still holding an unterminated message.

The operation that decides this is the server’s blocking call: it blocks in `kevent()` waiting for *readiness across all registered descriptors*, not in `recv()` on any one socket. `procstat -k` proves it — the only thread’s kernel stack is `... _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_kevent_generic sys_kevent ...`. The server *is* asleep, but inside `sys_kevent`. There is no `soreceive`, no `recv`, nothing tied to Client 1’s socket anywhere in the stack. When Client 2’s data arrives the kernel makes fd 6 ready, `kevent()` returns it, and the loop services it without revisiting fd 5.

A blocking server that called `recv(fd5)` in a fixed order would instead be parked in `soreceive` on fd 5 indefinitely, and Client 2 would never be accepted — precisely the stall this design avoids.

Where the incompleteness lives. Client 1’s Recv-Q is **0, not 20**. The server already performed `recv()` on fd 5 and received all 20 bytes (`[5] recv() -> 20 byte(s)`); they sit in that connection’s application-level `LineBuffer` awaiting a newline that never comes, so no command line is ever emitted for fd 5. The kernel socket is drained and quiescent. The partial message is therefore an *application-layer* state, not a TCP-layer one: TCP has delivered everything sent and has nothing outstanding.

Approach. Ran the harness, which makes Client 1 send a message with no terminator and then fall silent, and Client 2 send a complete message ≈ 2 s later. Measured the latency the harness reports for Client 2’s reply, then — while both connections were still open — inspected the server from outside: its kernel stack, the socket table, and its descriptor table.

Commands and tools.

```
python3 -u experiment.py 4 2>&1 | tee exp4_harness.txt
SRV=$(pgrep -x exchange_server); procstat -k $SRV | tee exp4_kstack.txt
netstat -an -p tcp | grep -E 'Proto|5000' | tee exp4_netstat.txt
sockstat -4 | grep -E 'USER|exchange' | tee exp4_sockstat.txt
```

`procstat -k` prints the in-kernel call stack of every thread; this is what identifies the blocking operation, which timing alone cannot.

```
vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && python3 -u experiment.py 4 2>&1 | tee ~/evidence/exp4/exp4_harness.txt
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 5947).
exchange_server listening on 127.0.0.1:5000 (kqueue fd=4, listen fd=3)
Exchange Server is listening.[+] accepted fd=5 from 127.0.0.1:65517 (1 client(s) connected)

[5] recv() -> ECONNRESET : peer reset (RST)
[-] fd=5 closed (0 client(s) still connected)
[+] accepted fd=5 from 127.0.0.1:62131 (1 client(s) connected)
Client 1: connected from 127.0.0.1:62131

Client 1 will send an incomplete application message.
Client 1 will now remain silent.

[5] recv() -> 20 byte(s)
Client 2: connected from 127.0.0.1:12998[+] accepted fd=6 from 127.0.0.1:12998 (2 client(s) connected)

Client 2 will send a complete application message.
[6] recv() -> 20 byte(s)
[6] LOGIN active_client

Client 2 response: 'OK'
Elapsed time: 0.001 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
```

Figure 13: Experiment 4 — `[5] recv() -> 20 byte(s)` with *no* parsed line following it (Client 1), versus `[6] recv() -> 20 -> LOGIN active_client -> OK` in 0.001 s (Client 2).

```

vzux@freebsd:~$ kill -x exchange_server ; sleep 1 ; (sockstat -4l | grep 5000) || echo "PORT FREE -- ok to start"
PORT FREE -- ok to start
vzux@freebsd:~$ SRV=$(pgrep -x exchange_server); echo "server pid $SRV"; procstat -k $SRV | tee -/evidence/exp4/exp4_kstack.txt
server pid 5947
PID TID COMM TONAME KSTACK
5947 100175 exchange_server mt_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_kevent_generic sys_kevent and64_syscall fast_syscall_common
vzux@freebsd:~$ netstat -an -p tcp | grep -E 'Proto|5000' | tee -/evidence/exp4/exp4_netstat.txt
Proto Recv-Q Send-Q Local Address Foreign Address (state)
tcp4 0 0 127.0.0.1:5000 127.0.0.1:12998 ESTABLISHED
tcp4 0 0 127.0.0.1:12998 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:62131 ESTABLISHED
tcp4 0 0 127.0.0.1:62131 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 *.* LISTEN
vzux@freebsd:~$ sockstat -4 | grep -E 'USER|exchange' | tee -/evidence/exp4/exp4_sockstat.txt
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
vzux exchange_s 5947 3 tcp4 127.0.0.1:5000 *.*
vzux exchange_s 5947 5 tcp4 127.0.0.1:5000 127.0.0.1:62131
vzux exchange_s 5947 6 tcp4 127.0.0.1:5000 127.0.0.1:12998
vzux@freebsd:~$

```

Figure 14: Experiment 4 — the decisive evidence. `procstat -k` shows the single thread asleep in `sys_kevent`, not in `soreceive`. Both connections ESTABLISHED with Recv-Q 0; `sockstat` shows fds 3/5/6 on one PID.

2.5 Experiment 5 — Multiple Clients and I/O Multiplexing

(optional experiment)

Question. At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence from the running system allows you to determine this?

Answer. Only the connections carrying inbound data were ready: Client 1 (fd 5, port 65198), Client 3 (fd 7, port 42987) and Client 5 (fd 9, port 42493). Clients 2 (fd 6) and 4 (fd 8) were fully ESTABLISHED but had sent nothing, so `kqueue` never reported them and the server never touched them.

Three independent lines of evidence establish this.

1. *The server's own trace.* The log contains `recv()` lines for fd 5, fd 7 and fd 9 only — 15 bytes each, each followed by the parsed LOGIN. There is **no [6] or [8] line anywhere**. The server did not call `recv()` on the idle sockets and get EAGAIN; it never called `recv()` on them at all. Readiness, not polling, decides what is serviced.

2. *netstat distinguishes them independently.* On the client side of each connection the three active clients show **Recv-Q = 3** — the server's 3-byte OK reply sitting unread — while the two idle clients show Recv-Q = 0 in both directions. Even without the server's log, the socket table alone identifies which connections saw application traffic.

3. *procstat shows the structural cost.* `procstat -f` lists one process holding fd 3 (listening), fd 4 (the `kqueue`) and fds 5–9 (the five connections). `procstat -k` shows exactly **one thread**, asleep in `kqueue_scan / sys_kevent`.

The mechanism: all five sockets are registered with `EVFILT_READ` on the same `kqueue`, and `kevent()` returns only descriptors the kernel has marked ready. The loop's work is proportional to the number of *active* connections, not *open* ones. Idle connections cost a descriptor and some kernel socket state, but no CPU — the property that makes the bonus tractable.

Approach. Ran the harness, which opens five connections and then has clients 1, 3 and 5 send a LOGIN 0.5s apart while 2 and 4 stay silent. During the 15-second observation window, captured three views of the live server: the socket table, its file-descriptor table, and its thread kernel stacks. Cross-referenced the fd numbers in the server's accept log against the ports in `netstat` to attribute each socket to a specific client.

Commands and tools.

```

python3 -u experiment.py 5 2>&1 | tee exp5_harness.txt
netstat -an -p tcp | grep -E 'Proto|5000' | tee exp5_netstat.txt
procstat -f $(pgrep -x exchange_server) | tee exp5_fds.txt
procstat -k $(pgrep -x exchange_server) | tee exp5_kstack.txt

```

```

vzux@Freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && python3 -u experiment.py 5 2>&1 | tee ~/evidence/exp5/exp5_harness.txt
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===

Started Exchange Server (PID 12903).
exchange_server listening on 127.0.0.1:5000 (kqueue fd=1, listen fd=3)
[+] accepted fd=5 from 127.0.0.1:165190 (1 client(s) connected)
Exchange Server is listening.[5] recv() -> ECONNRESET : peer reset (RST)
[-] fd=5 closed (0 client(s) still connected)

Creating five simultaneous TCP connections...
Client 1: connected from 127.0.0.1:165198
Client 2: connected from 127.0.0.1:143356
[+] accepted fd=6 from 127.0.0.1:165190 (1 client(s) connected)
[+] accepted fd=6 from 127.0.0.1:143356 (2 client(s) connected)
Client 3: connected from 127.0.0.1:142987
Client 4: connected from 127.0.0.1:161690
[+] accepted fd=7 from 127.0.0.1:142987 (3 client(s) connected)
[+] accepted fd=8 from 127.0.0.1:161690 (4 client(s) connected)
[+] accepted fd=9 from 127.0.0.1:142983 (5 client(s) connected)
Client 5: connected from 127.0.0.1:142493

Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
[5] recv() -> 15 byte(s)
[5] LOGIN client_1
Client 3: sent application data.
[7] recv() -> 15 byte(s)
[7] LOGIN client_3
Client 5: sent application data.
[9] recv() -> 15 byte(s)
[9] LOGIN client_5

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
vzux@Freebsd:~/A2_ROLL1_ROLL2 $

```

Figure 15: Experiment 5 — five accepted `fd=` lines mapping `fd` to port, then `recv()`/`LOGIN` for clients 1, 3 and 5 and no [6] or [8] lines at all.

```

vzux@Freebsd:~$ skill -x exchange_server ; sleep 1 ; (sockstat -tl | grep 5000) || echo "PORT FREE -- ok to start"
PORT FREE -- ok to start
vzux@Freebsd:~$ netstat -an -p tcp | grep -E "Proto5000" | tee ~/evidence/exp5/exp5_netstat.txt
Proto Recv-Q Send-Q Local Address Foreign Address (state)
tcp4 0 0 127.0.0.1:142493 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:161690 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:142987 ESTABLISHED
tcp4 3 0 127.0.0.1:142987 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:143356 ESTABLISHED
tcp4 0 0 127.0.0.1:143356 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:165190 ESTABLISHED
tcp4 3 0 127.0.0.1:165190 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 *.* LISTEN
vzux@Freebsd:~$ procstat -f $(pgrep -x exchange_server) | tee ~/evidence/exp5/exp5_fds.txt
PID COMM FD T V FLAGS REF OFFSET PRO NAME
12903 exchange_server text v r r - - - - /home/vzux/A2_ROLL1_ROLL2/bin/exchange_server
12903 exchange_server cwd v d r - - - - /home/vzux/A2_ROLL1_ROLL2
12903 exchange_server root v d r - - - - /
12903 exchange_server 0 v c rw - - - 7 1044271 - /dev/pts/1
12903 exchange_server 1 0 - rw - - - 4 0 -
12903 exchange_server 2 p - rw - - - 4 0 -
12903 exchange_server 3 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 *:0
12903 exchange_server 4 k - rw - - - 2 0 -
12903 exchange_server 5 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:165190
12903 exchange_server 6 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:143356
12903 exchange_server 7 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:142987
12903 exchange_server 8 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:161690
12903 exchange_server 9 s - rw - - - 2 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:142493
vzux@Freebsd:~$ procstat -k $(pgrep -x exchange_server) | tee ~/evidence/exp5/exp5_kstack.txt
PID TID COMM TDNAME KSTACK
12903 100172 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_kevent_generic sys_kevent and64_syscall fast_syscall_common
vzux@Freebsd:~$

```

Figure 16: Experiment 5 — `netstat` (Recv-Q 3 on the three active clients versus 0 on the idle two), `procstat -f` (fd 3 listen, fd 4 kqueue, fds 5–9 connections on one PID), and `procstat -k` (one thread in `sys_kevent`).

2.6 Experiment 6 — FIN vs. RST

Question. How does an abrupt client termination differ from the orderly shutdown? Identify the TCP event observed on the network and the corresponding behaviour of the Exchange Server’s socket.

Answer. The two differ in both what crosses the wire and what the server’s socket reports, and the difference is *negotiated* versus *unilateral*.

Orderly (Part A, client 127.0.0.1:33130) — 7 packets. The client calls `shutdown()` on its sending direction, putting a FIN on the wire. The close is a negotiated four-packet exchange: client FIN [F.] → server ACK [.] → server FIN [F.] → client ACK [..]. Each FIN consumes a sequence number and is acknowledged. On the server the FIN surfaces as `recv()` **returning 0** — a normal end-of-stream, not an error — logged as [5] `recv() -> 0 : peer closed (FIN)`. The server then closes its own side, emitting its FIN 142 μ s later.

Abortive (Part B, client 127.0.0.1:25854) — 4 packets. The client sets `SO_LINGER` with a zero timeout and closes, which makes the kernel send RST instead of FIN. This is unilateral: **a single [R.] packet, never acknowledged**, with no reply from the server and no further exchange of any kind. On the server the reset surfaces as `recv()` **failing with ECONNRESET**, logged as [5] `recv() -> ECONNRESET : peer reset (RST)`. Any queued outbound data would be discarded rather than delivered, and a subsequent `send()` would fail with `EPIPE`.

So the distinguishing TCP event is the flag itself — FIN versus RST — and the corresponding socket behaviour is `recv() = 0` (orderly EOF) versus `recv() = -1` with `ECONNRESET` (error). The packet counts, 7 versus 4, quantify it: RST needs no acknowledgement and no second direction, which is exactly why it is “abrupt”.

RST arises from two distinct causes in this capture. At 16:17:20.509266 the server side sends [R.] in reply to a SYN on port 5000 *before the server had bound*: a SYN to a port with no listener is answered with RST (connection refused), never with FIN. That is a different cause from Part B’s deliberate abortive close at .678706, yet both produce the same single unacknowledged reset packet. A third RST at .610374 is the harness’s readiness probe closing abortively after confirming the server was up. Only .678706 is the experiment’s own Part B reset.

Note on netstat versus the capture. The 1-second sampler never caught ESTABLISHED for Part A at all — the client shut down 274 μ s after connecting, far below the sampling interval. What the sampler *did* show is the client socket sitting in state CLOSED from 16:17:21 to 16:17:29, disappearing at 16:17:30. The TCP connection was fully torn down at .611, but the harness had not yet called `close()` on the file descriptor, so the socket still existed — separating TCP *connection* state from *socket/fd* lifetime. (This is not itself a FIN-versus-RST difference: Part B’s fd was closed immediately, so the two are not directly comparable on that point.)

Approach. Ran the harness, which performs an orderly `shutdown()` first and an `SO_LINGER{1,0}` abortive close second, with a packet capture and a 1-second state sampler running throughout. Compared three views of the same two events: the flags on the wire, the server’s own logging of what its `recv()` call returned, and the socket table. Finally filtered the capture down to only FIN and RST lines so the two terminations sit side by side.

Commands and tools.

```
su root -c 'tcpdump -l -i lo0 -n -tttt -S "tcp port 5000"' | tee exp6_tcpdump.txt
while true; do date +%T; netstat -an -p tcp | grep 5000; echo; sleep 1; done \
    | tee exp6_states.txt
python3 -u experiment.py 6 2>&1 | tee exp6_harness.txt
grep -E 'Flags \[(F|R)\' exp6_tcpdump.txt | tee exp6_fin_rst_only.txt
```

-S prints absolute sequence numbers so the FIN’s sequence consumption is visible. The server dis-

tinguishes the two paths explicitly at `exchange_server.cpp:382` (`recv()==0`) and `:393` (`ECONNRESET`).

```
vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && python3 -u experiment.py 6 2->81 | tee ~/evidence/exp6/exp6_harness.txt
=== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ===

Started Exchange Server (PID 6055).
exchange_server listening on 127.0.0.1:5000 (kqueue fd=4, listen fd=3)
[+] accepted fd=5 from 127.0.0.1:60412 (1 client(s) connected)
[5] recv() -> ECONNRESET : peer reset (RST)
Exchange Server is listening.[+] fd=5 closed (0 client(s) still connected)

Part A: orderly connection termination (FIN).
[+] accepted fd=5 from 127.0.0.1:33130 (1 client(s) connected)
Orderly client: connected from 127.0.0.1:33130
The client will shut down its sending direction.
Investigate the TCP traffic and connection state.
[5] recv() -> 0 : peer closed (FIN)
[-] fd=5 closed (0 client(s) still connected)

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:25854
The client will now close abortively.
[+] accepted fd=5 from 127.0.0.1:25854 (1 client(s) connected)
Investigate the TCP traffic and connection state.

[5] recv() -> ECONNRESET : peer reset (RST)
The Exchange Server will remain running for 15 seconds.
[-] fd=5 closed (0 client(s) still connected)

Experiment finished.
Stopping Exchange Server...
vzux@freebsd:~/A2_ROLL1_ROLL2 $
```

Figure 17: Experiment 6 — the application-visible difference: Part A `recv() -> 0 : peer closed (FIN)` versus Part B `recv() -> ECONNRESET : peer reset (RST)`.

```
vzux@freebsd:~$ su root -c 'tcpdump -l -i lo0 -n -tttt -S "tcp port 5000" | tee ~/evidence/exp6/exp6_tcpdump.txt
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
2026-09-09 16:17:20.507024 IP 127.0.0.1.48656 > 127.0.0.1.5000: Flags [S], seq 571499978, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 455215490 ecr 0], length 0
2026-09-09 16:17:20.509266 IP 127.0.0.1.5000 > 127.0.0.1.48656: Flags [R..], seq 0, ack 571499979, win 0, length 0
2026-09-09 16:17:20.609812 IP 127.0.0.1.60412 > 127.0.0.1.5000: Flags [S..], seq 129271317, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 871472207 ecr 0], length 0
2026-09-09 16:17:20.609969 IP 127.0.0.1.5000 > 127.0.0.1.60412: Flags [S..], seq 1636960407, ack 129271318, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3696986480 ecr 871472207], length 0
2026-09-09 16:17:20.610083 IP 127.0.0.1.60412 > 127.0.0.1.5000: Flags [..], ack 1636960408, win 1277, options [nop,nop,TS val 871472207 ecr 3696986480], length 0
2026-09-09 16:17:20.610374 IP 127.0.0.1.60412 > 127.0.0.1.5000: Flags [R..], seq 129271319, ack 1636960408, win 0, options [nop,nop,TS val 871472207 ecr 3696986480], length 0
2026-09-09 16:17:20.610775 IP 127.0.0.1.33130 > 127.0.0.1.5000: Flags [S..], seq 3669074551, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3547807222 ecr 0], length 0
2026-09-09 16:17:20.610842 IP 127.0.0.1.5000 > 127.0.0.1.33130: Flags [S..], seq 3013434693, ack 3669074552, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3319884344 ecr 3547807222], length 0
2026-09-09 16:17:20.610863 IP 127.0.0.1.33130 > 127.0.0.1.5000: Flags [..], ack 3013434694, win 1277, options [nop,nop,TS val 3547807222 ecr 3319884344], length 0
2026-09-09 16:17:20.611137 IP 127.0.0.1.33130 > 127.0.0.1.5000: Flags [F..], seq 3669074552, ack 3013434694, win 1277, options [nop,nop,TS val 3547807222 ecr 3319884344], length 0
2026-09-09 16:17:20.611152 IP 127.0.0.1.5000 > 127.0.0.1.33130: Flags [..], ack 3669074553, win 1277, options [nop,nop,TS val 3319884344 ecr 3547807222], length 0
2026-09-09 16:17:20.611279 IP 127.0.0.1.5000 > 127.0.0.1.33130: Flags [F..], seq 3013434694, ack 3669074553, win 1277, options [nop,nop,TS val 3319884344 ecr 3547807222], length 0
2026-09-09 16:17:20.611286 IP 127.0.0.1.33130 > 127.0.0.1.5000: Flags [..], ack 3013434695, win 1277, options [nop,nop,TS val 3547807222 ecr 3319884344], length 0
2026-09-09 16:17:30.678451 IP 127.0.0.1.25854 > 127.0.0.1.5000: Flags [S..], seq 391889474, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4281584880 ecr 0], length 0
2026-09-09 16:17:30.678482 IP 127.0.0.1.5000 > 127.0.0.1.25854: Flags [S..], seq 1958753155, ack 391889475, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3256429753 ecr 4281584880], length 0
2026-09-09 16:17:30.678510 IP 127.0.0.1.25854 > 127.0.0.1.5000: Flags [..], ack 1958753156, win 1277, options [nop,nop,TS val 4281584880 ecr 3256429753], length 0
2026-09-09 16:17:30.678706 IP 127.0.0.1.25854 > 127.0.0.1.5000: Flags [R..], seq 391889475, ack 1958753156, win 0, options [nop,nop,TS val 4281584880 ecr 3256429753], length 0
^C17 packets captured
17 packets received by filter
0 packets dropped by kernel
vzux@freebsd:~$
```

Figure 18: Experiment 6 — the complete 17-packet capture covering both terminations.

```
vzux@freebsd:~$ grep -E 'Flags \[([FR])\]' ~/evidence/exp6/exp6_tcpdump.txt | tee ~/evidence/exp6/exp6_fin_rst_only.txt
2026-09-09 16:17:20.509266 IP 127.0.0.1.5000 > 127.0.0.1.48656: Flags [R..], seq 0, ack 571499979, win 0, length 0
2026-09-09 16:17:20.610374 IP 127.0.0.1.60412 > 127.0.0.1.5000: Flags [S..], seq 129271317, win 65535, options [nop,nop,TS val 871472207 ecr 3696986480], length 0
2026-09-09 16:17:20.611137 IP 127.0.0.1.33130 > 127.0.0.1.5000: Flags [F..], seq 3669074552, ack 3013434694, win 1277, options [nop,nop,TS val 3547807222 ecr 3319884344], length 0
2026-09-09 16:17:20.611279 IP 127.0.0.1.5000 > 127.0.0.1.33130: Flags [F..], seq 3013434694, ack 3669074553, win 1277, options [nop,nop,TS val 3319884344 ecr 3547807222], length 0
2026-09-09 16:17:30.678706 IP 127.0.0.1.25854 > 127.0.0.1.5000: Flags [R..], seq 391889475, ack 1958753156, win 0, options [nop,nop,TS val 4281584880 ecr 3256429753], length 0
vzux@freebsd:~$
```

Figure 19: Experiment 6 — only the FIN and RST lines: two FINs (Part A) and three RSTs (connection-refused, harness probe, and Part B's abortive close).

16:17:21					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:22					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:23					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:24					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:25					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:26					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:27					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:28					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:29					
tcp4	0	0	127.0.0.1.33130	127.0.0.1.5000	CLOSED
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:30					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:31					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:32					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:33					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:34					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:35					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:37					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:38					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:39					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:40					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:41					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:42					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:43					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:44					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:45					
tcp4	0	0	127.0.0.1.5000	*,*	LISTEN
16:17:46					
16:17:47					

Figure 20: Experiment 6 — the client socket in state CLOSED from 16:17:21 to 16:17:29, gone at :30 when the fd is finally closed.

2.7 Experiment 7 — Backpressure and the Slow Receiver

Question. How does the Exchange Server’s TCP connection to the slow client behave as the client stops reading, and what evidence shows whether this eventually affects the server’s ability to communicate with other clients?

Answer. The connection to the slow client stays **ESTABLISHED** throughout and never errors, but undelivered market data accumulates in its kernel receive buffer, climbing monotonically from 0 to a hard ceiling of **85,000 bytes**. Over the same period the normal Market-Data client’s Recv-Q never rose above 17 bytes and both Trader connections kept exchanging orders continuously, with all 5,000 trades completing. The slow reader therefore did **not** affect the server’s ability to serve anyone else — the backlog is confined to its own socket.

The mechanism is TCP flow control plus per-connection output queuing. As the slow client stops calling `recv()`, its receive buffer fills and the window it advertises shrinks. The server never blocks on this, because it holds pending output in a per-connection application-level queue and only calls `send()` when `kqueue` reports that socket writable via `EVFILT_WRITE`. Sockets that are not writable are simply not serviced that iteration; every other descriptor continues to be.

Why Send-Q stays near zero — the key subtlety. `netstat`’s Send-Q shows only the *kernel* socket buffer. Peak server-side Send-Q toward the slow client was 34 bytes (Pass A) and 51 bytes (Pass B) — essentially nothing. That is not evidence that backpressure is absent; it is evidence that the server never over-commits to a socket the kernel will not accept. The backlog lives in the server’s own userspace output queue, which `netstat` cannot see. A naive server calling a blocking `send()` in a loop would instead fill the kernel buffer, then block inside `send()` on the slow client’s socket, and every other client would stall behind it. The near-zero Send-Q is the design working, not failing.


```
[7] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
[7] recv() -> 15 byte(s)
[7] BUY_NMSG 1 238
[8] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
[7] recv() -> 15 byte(s)
[7] BUY_NMSG 1 238
[8] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
[7] recv() -> 15 byte(s)
[7] BUY_NMSG 1 238
[8] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
[7] recv() -> 15 byte(s)
[7] BUY_NMSG 1 238
[8] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
[7] recv() -> 15 byte(s)
[7] BUY_NMSG 1 238
[8] recv() -> 16 byte(s)
[8] SEND_NMSG 1 238
```

Generated 5000 matching trades.
The slow client has intentionally not consumed its data.

Investigate both Market-Data TCP connections.
Press Ctrl-C when you are finished.

```
[5] recv() -> 0 : peer closed (FIN)
[-] fdm closed (2 client(s) still connected)
[6] recv() -> 0 : peer closed (FIN)
Stopping Exchange Server...
[-] fdm closed (2 client(s) still connected)
[7] recv() -> 0 : peer closed (FIN)
[-] fdm closed (1 client(s) still connected)
[8] recv() -> 0 : peer closed (FIN)
[-] fdm closed (0 client(s) still connected)
```

```
xvzfzfreedbi:/AZ_ROLL1_ROLL_5 grep -E "Market-Data Client:|Trader:" --jwldence/exp7/exp7B_harness.txt
normal Market-Data Client: connected from 127.0.0.1:144850
slow Market-Data Client: connected from 127.0.0.1:34500
Buyer Trader: connected from 127.0.0.1:39712
xvzfzfreedbi:/AZ_ROLL1_ROLL_5
xvzfzfreedbi:/AZ_ROLL1_ROLL_5 accepted fdm from 127.0.0.1:15058 [-] accepted fdm from 127.0.0.1:15058 (4 client(s) connected)
```

Figure 26: Experiment 7, Pass B — Generated 5000 matching trades. confirming the run completed with auto-tuning disabled, plus the clean shutdown and role mapping.

```
vzx@freedsb:~$ grep -E "Market-Data Client|Trader:" -r /evidence/exp7/exp7b_harness.txt
Normal Market-Data Client: connected from 127.0.0.1:44850
Slow Market-Data Client: connected from 127.0.0.1:34500
Buyer Trader: connected from 127.0.0.1:39712
Seller Trader: connected from 127.0.0.1:15058 [+] accepted fd=8 from 127.0.0.1:15058 (4 client(s) connected)
vzx@freedsb:~$
```

Figure 27: Experiment 7, Pass B — role-to-port mapping.

2.8 Experiment 8 — Unexpected Client Disconnection

Question. When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and what network-level evidence allows you to determine how the server detects the failure?

Answer. The connection is closed **gracefully**, and the server detects the failure exactly as it would detect a polite disconnect: `recv()` returns 0.

The disappearing client was a *separate process* (server fd 8, 127.0.0.1:41329) killed with SIGKILL, so it had no opportunity to run shutdown code, close its socket, or send anything deliberately. Nevertheless a normal **FIN** appeared on the wire at 17:01:09.148094, and the teardown was the full four-way exchange: client FIN [F.] → server ACK (.148173) → server FIN (.148281) → client ACK (.148290). The server logged [8] `recv() -> 0 : peer closed (FIN)` and closed fd 8.

The reason is that **socket teardown is the kernel’s responsibility, not the application’s**. When a process dies — however violently — the kernel reclaims its file descriptors, and closing a TCP socket emits FIN. So an abrupt *process* failure still produces an orderly *connection* close, and at the socket layer it is indistinguishable from the client calling `close()` itself. This is the key contrast with Experiment 6, where RST was produced by `SO_LINGER{1,0}` — an explicit application-level choice about *how* to close. A crash makes no such choice, so it produces no RST.

Notably, **306 bytes** of undelivered market data were still sitting unread in that client's receive buffer at 17:01:08 (the helper process was parked in `sleep()`, not reading), and FreeBSD still sent FIN rather than a reset.

RST would arise only afterwards, if the server sent further data to the dead socket — the peer’s kernel, no longer holding that four-tuple, would answer with RST. That did not happen here because the event loop saw `recv() = 0` immediately and closed its own side within $187\ \mu\text{s}$, before further market data could be written.

The other clients were completely unaffected. In the same 1-second sample where the dead

connection vanished (17:01:09), the surviving Market-Data client (57334) and both Traders (53315, 29852) remained ESTABLISHED, and the capture shows trade traffic to 29852 and 53315 flowing at 17:01:09.077 before the FIN and resuming at .149233 immediately after. The server states it plainly: [-] fd=8 closed (3 client(s) still connected).

Approach. Ran the harness, which starts a genuinely separate client process, generates real trades so the server is actively communicating with it, then SIGKILLS that process while a second Market-Data client and two Traders stay connected. Observed the packets around the kill, the socket table before and after, and the server’s own report of what its `recv()` returned. Because the killed client is a separate process the harness never prints its port, so the port had to be recovered from the server’s accept log.

Commands and tools.

```
su root -c 'tcpdump -l -i lo0 -n -tttt "tcp port 5000" | tee exp8_tcpdump.txt
while true; do date +%T; netstat -an -p tcp | grep 5000; echo; sleep 1; done \
    | tee exp8_states.txt
python3 -u experiment.py 8 2>&1 | tee exp8_harness.txt
grep -E "Market-Data Client|Trader:|accepted fd=" exp8_harness.txt
```

fd	Port	Role	Fate
5	57334	Surviving Market-Data	survives
6	53315	Buyer Trader	survives
7	29852	Seller Trader	survives
8	41329	Disappearing Market-Data	SIGKILLed

```
[7] SELL JNST 1 238  
[6] recv() -> 15 byte(s)  
[6] BUY JNST 1 238  
[7] recv() -> 16 byte(s)  
[7] SELL JNST 1 238  
[6] recv() -> 15 byte(s)  
[6] BUY JNST 1 238  
[7] recv() -> 16 byte(s)  
[7] SELL JNST 1 238  
[6] recv() -> 15 byte(s)  
[6] BUY JNST 1 238  
[7] recv() -> 16 byte(s)  
[7] SELL JNST 1 238  
[6] recv() -> 15 byte(s)  
[6] BUY JNST 1 238  
[7] recv() -> 16 byte(s)  
[7] SELL JNST 1 238
```

Experiment finished.

```
[5] recv() -> 0 : peer closed (FIN)
```

Stopping Exchange Server...

```
[1] fdns closed (2 client(s) still connected)  
[6] recv() -> 0 : peer closed (FIN)  
[1] fdns closed (4 client(s) still connected)  
[7] recv() -> 0 : peer closed (FIN)  
[1] fdns closed (8 client(s) still connected)
```

vixurfreesdb:/AZ_ROLLI_ROLL2 \$ grep -E "Market-Data Client|Trader:accepted fd=" ~/evidence/exp8/exp_harness.txt

Exchange Server is listening [4] accepted fd=5 from 127.0.0.1:25368 (1 client(s) connected)

```
[+] accepted fds from 127.0.0.1:157334 (1 client(s) connected)  
Surviving Market-Data Client: connected from 127.0.0.1:157334  
[+] accepted fds from 127.0.0.1:153315 (2 client(s) connected)  
Buyer Trader: connected from 127.0.0.1:153315  
Seller Trader: connected from 127.0.0.1:29852[+] accepted fd=7 from 127.0.0.1:29852 (3 client(s) connected)  
[+] accepted fds from 127.0.0.1:41329 (4 client(s) connected)
```

The Market-Data Clients should now be connected.

The Market-Data Client process will now disappear unexpectedly.

vixurfreesdb:/AZ_ROLLI_ROLL2 \$ sed -n '109,110p' ~/evidence/exp8/exp_harness.txt

The Market-Data client process will now disappear unexpectedly.

```
[8] recv() -> 0 : peer closed (FIN)  
[1] fdns closed (3 client(s) still connected)
```

The client process has disappeared.

Generating additional traffic.

Investigate the terminated connection and the surviving one.

Press Ctrl-C when you are finished.

```
[6] recv() -> 15 byte(s)  
vixurfreesdb:/AZ_ROLLI_ROLL2 $
```

Figure 28: Experiment 8 — fd-to-port mapping including accepted fd=8 from 127.0.0.1:41329, then the kill moment with [8] recv() -> 0 : peer closed (FIN) and 3 client(s) still connected.

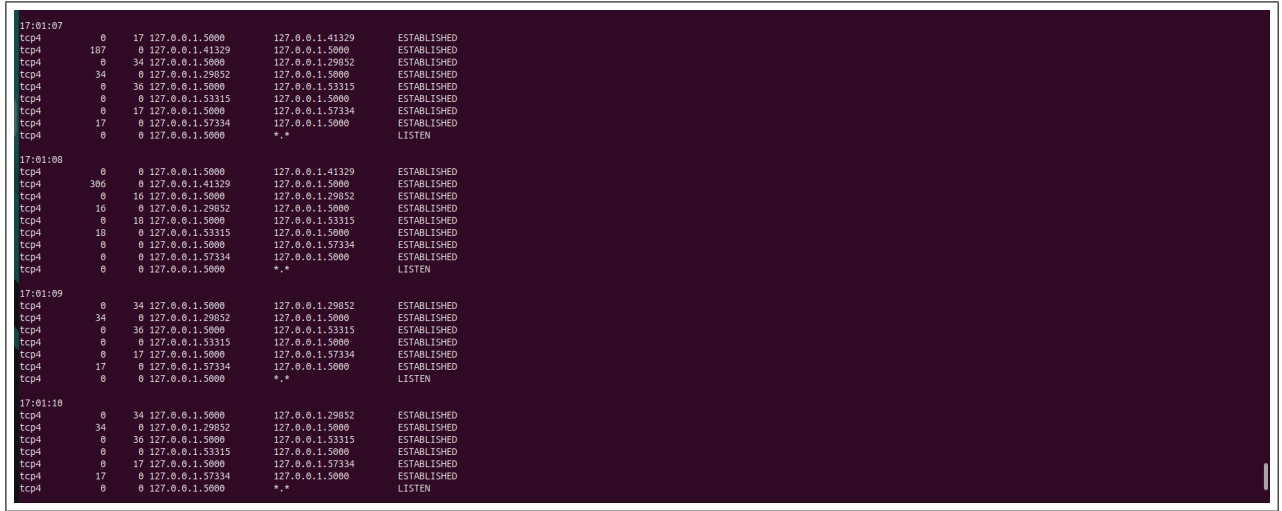


Figure 29: Experiment 8 — 41329 present with Recv-Q 187 → 306 at 17:01:07–08, gone at 17:01:09, while the other three remain ESTABLISHED.

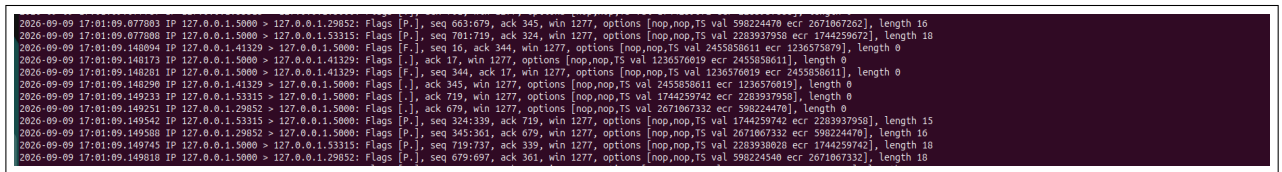


Figure 30: Experiment 8 — the complete four-way close of 41329, sandwiched between trade traffic to 29852/53315 before and after: one frame proving the other clients kept flowing throughout.

3 Bonus — Connection Scalability and I/O Design

3.1 Client-generation program

`bonus/conn_generator.cpp` opens and holds N idle TCP connections, exchanging no application data. It raises its own `RLIMIT_NOFILE` to the hard limit, optionally `bind()`s each client socket round-robin across a list of source addresses (see §3.4), reports progress every 5,000 connections, and on failure prints the exact `errno` together with an interpretation. Helper scripts `bonus/bonus_run.sh` (runs one level end-to-end) and `bonus/bonus_measure.sh` (collects every table column in one pass) accompany it.

3.2 Resource-measurement table

All measurements were taken while the stated number of connections were simultaneously ESTABLISHED and idle.

Idle conns	Server RSS	CPU	Server FDs	System-wide FDs	Sock-buf	Max established
10,000	5,468 KB	0.0%	10,009	20,102 / 400,000	0 / 251,575	10,000
20,000	7,432 KB	0.0%	20,008	40,115 / 400,000	0 / 251,575	20,000
30,000	9,524 KB	0.0%	30,008	60,105 / 400,000	0 / 251,575	30,000
40,000	11,396 KB	0.0%	40,009	80,102 / 400,000	0 / 251,575	40,000
50,000	13,300 KB	0.0%	50,008	100,105 / 400,000	0 / 251,575	50,000
60,000	15,616 KB	0.1%	60,008	120,104 / 400,000	0 / 251,575	60,000
70,000	17,532 KB	0.1%	70,009	140,102 / 400,000	0 / 251,575	70,000

Reading the table. Server FDs = connections + 3 (listening socket, kqueue, stdio). System-wide FDs

= exactly $2 \times$ connections + ≈ 105 : both endpoints are local, so every connection consumes **two** file table entries and **two** socket structures. This factor of two governs every system-wide limit in the experiment. Sock-buf is mbuf *clusters* in use from `netstat -m`; it is zero at every level (see Q3). The 10k/40k/70k rows are the values in the submitted screenshots; the others come from separate runs of the same script, where run-to-run variation is a few KB of RSS.

3.3 Q1 — Able to maintain all requested connections?

Yes at every level from 10,000 to 70,000 inclusive, *after* tuning; all seven runs reported `SUCCESS: all N connections established`.

Before tuning it failed at **64,909 of 70,000** with `errno 55 (ENOBUFFS, “No buffer space available”)`. That is the first-failure point on a default-configured guest; diagnosing it is Q2.

3.4 Q2 — First significant bottleneck, with supporting data

The kernel’s socket UMA zone — not memory, CPU, file descriptors or mbufs. From `vmstat -z` at the moment of failure:

ITEM	SIZE	LIMIT	USED	FREE	REQ	FAIL
socket:	1024,	129967,	62006,	67962,	214502,	110

- LIMIT was 129,967 — identical to the guest’s *boot-time* `kern.maxfiles`.
- FAIL 110 is the kernel’s own count of refused allocations from that zone; the generator observed those same refusals as `ENOBUFFS`.
- Because both endpoints are local, N connections need $2N$ socket structures, so the zone caps connections at $129,967/2 = 64,983$. The run stopped at **64,909** — within 0.1% of the prediction.

Everything else had large headroom at that moment, which is what rules the alternatives out: server RSS 16.2 MB of 4 GB (0.4%); 130,006 open files of `kern.maxfiles` 400,000 (32.5%); 64,912 fds of `kern.maxfilesperproc` 200,000; 0 mbuf clusters of 251,575; CPU 0.0% with 0.34s of CPU time accumulated in total.

The non-obvious part. Raising `kern.maxfiles` from 129,967 to 400,000 at runtime did *not* help, because the socket zone is sized once at boot from that value. The tunable that actually gates socket allocation is `kern.ipc.maxsockets`, which *is* writable at runtime and does resize the zone:

```
sysctl kern.ipc.maxsockets=400000      # socket zone LIMIT 129967 -> 400000
```

A second limit was anticipated and designed around rather than hit. A TCP connection is identified by (src IP, src port, dst IP, dst port). Every connection here targets the same 127.0.0.1:5000, so three elements are fixed and uniqueness rests on the source port alone — a 16-bit value. `net.inet.ip.portrange` defaults to 10000–65535 = 55,536 ports, and even widened to 1024–65535 gives only 64,512. **70,000 is therefore arithmetically unreachable from a single source address.** The generator binds each client socket round-robin across three loopback aliases (127.0.0.1, 127.0.0.2, 127.0.0.3, added with `ifconfig lo0 alias`), giving $3 \times 64,512 = 193,536$ of tuple space. The server’s accept log confirms all three addresses in use throughout the run.

A third, softer symptom appeared at $\approx 65,000$ connections: the guest stopped accepting new inbound SSH sessions entirely, and recovered the instant the connections were released. Existing sessions were unaffected.

3.5 Q3 — How the concurrency/I/O design contributes

The server is a single-threaded `kqueue()` event loop; it creates no thread and no process per connection, and `procstat -k` confirms one thread throughout (Experiments 4 and 5).

The consequence is visible directly in the table. Server RSS rises from 5,468 KB at 10,000 connections to 17,532 KB at 70,000 — a marginal cost of $\approx 2,012$ KB per 10,000 connections, i.e. **about 201 bytes per idle connection**, on a 3.4 MB baseline, and the relationship is linear across all seven levels. CPU stays at 0.0–0.1% and accumulates 0.33s while holding 70,000 connections, because idle sockets are never ready and so are never visited by the loop.

The design therefore contributes to the bottleneck only indirectly: it pushes per-connection application cost so low that the application is nowhere near being the limit, and what fails first is a *kernel* allocator ceiling. That is the intended outcome — the wall is an OS resource, not the program.

Supporting finding. mbuf cluster usage is *identical* (320/696/1016 4k jumbo clusters) at 1,000 connections and at 70,005. Idle sockets allocate **no socket buffer memory at all**; `sendspace` and `recvspace` are ceilings applied as data arrives, not reservations made at connect time. This is the same property that kept Send-Q near zero in Experiment 7, observed from the opposite direction.

3.6 Q4 — Would changing the I/O mechanism help?

No — every alternative is worse, and the measurements say why.

Thread-per-connection: 70,000 threads, each with its own stack. Even at a conservative 512 KB per stack that is ≈ 34 GB of address space against 4 GB of RAM; at FreeBSD’s default it is far more. The measured per-connection cost here is ≈ 201 bytes — roughly four orders of magnitude smaller. Thread-per-connection could not reach 70,000 on this machine at any tuning, and would add context-switch and scheduler cost that `kqueue` avoids entirely.

select(): bounded by `FD_SETSIZE` (1024 by default) and requires rebuilding and scanning the descriptor set on every call. Unusable at this scale.

poll(): no `FD_SETSIZE` limit, but the caller passes the entire array on every call and the kernel scans all of it — $O(n)$ per iteration in the number of *open* descriptors. At 70,000 mostly-idle connections that is 70,000 entries examined to find the handful that are ready. `kqueue` is $O(\text{ready})$: registration happens once, and `kevent()` returns only what changed.

Since the observed limit is a kernel socket-zone ceiling rather than anything the application controls, no change of I/O mechanism would raise it. Only the `sysctl` does; changing the mechanism could only make the application side worse.

3.7 Q5 — Quantified trade-off after a change

One configuration variable changed, everything else held constant:

<code>kern.ipc.maxsockets</code>	socket zone LIMIT	Max connections	Result
129,967 (boot value)	129,967	64,909	ENOBUFFS at 64,909
400,000	400,000	70,005	SUCCESS, headroom remains

Raising the zone from 129,967 to 400,000 lifted the achievable connection count from 64,909 to at least 70,005 — an 8% increase, for a change that costs no memory until the sockets are actually created. At 70,005 connections the zone held 140,104 of 400,000 entries (35%), so the new ceiling is roughly 200,000 connections: the limit has moved well beyond the assignment target.

The trade-off is that the zone is a pre-declared kernel ceiling — raising it permits more kernel memory to be committed to sockets if they are created. The measured cost of actually reaching

70,000 was 17.1 MB of server RSS and 140,102 file table entries, both trivial against 4 GB of RAM, which is why the increase is essentially free on this system.

A second change was considered and rejected on evidence: reducing `net.inet.tcp.sendspace/recvspace` to shrink per-connection buffer memory. The measurements show idle sockets allocate zero mbuf clusters, so that change would have measured no difference. It is recorded here because ruling it out is what established that socket-buffer memory is not on the critical path.

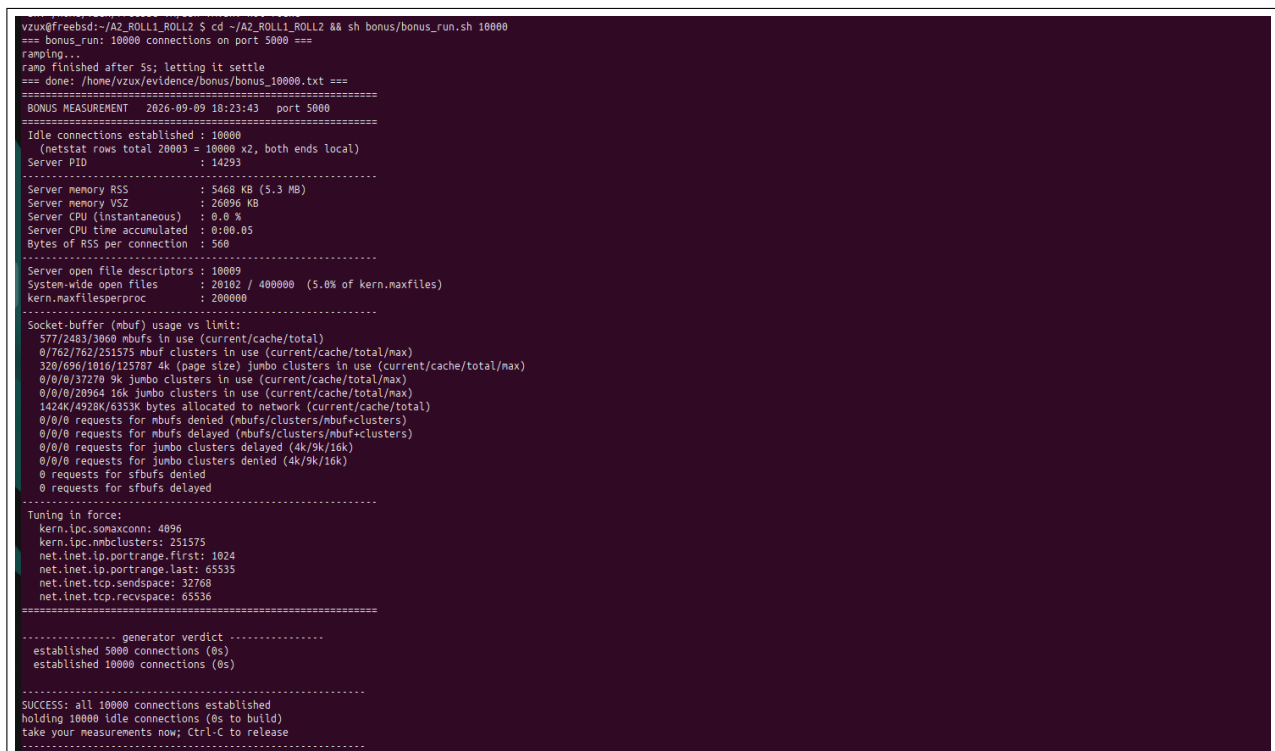
3.8 Reproducing the tuning

Runtime-only; re-apply after any reboot (as root):

```
sysctl kern.ipc.maxsockets=400000    # the important one -- sizes the socket zone
sysctl kern.maxfiles=400000
sysctl kern.maxfilesperproc=200000
sysctl kern.ipc.somaxconn=4096
sysctl net.inet.ip.portrange.first=1024
ifconfig lo0 alias 127.0.0.2/32
ifconfig lo0 alias 127.0.0.3/32
```

Then: `sh bonus/bonus_run.sh <count>` for one level, or `sh bonus/bonus_measure.sh 5000` to measure a running server.

3.9 Bonus screenshots



```
vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && sh bonus/bonus_run.sh 10000
=== bonus_run: 10000 connections on port 5000 ===
ramping...
ramp finished after 5s; letting it settle
=== done: /home/vzux/evidence/bonus/bonus_10000.txt ===
=====
BONUS MEASUREMENT  2026-09-09 18:23:43  port 5000
=====
Idle connections established: 10000
(netstat rows total 20003 = 10000 x2, both ends local)
Server PID          : 14293
-----
Server memory RSS    : 5468 KB (5.3 MB)
Server memory VSZ    : 26096 KB
Server CPU (instantaneous) : 0.0 %
Server CPU time accumulated : 0:00.05
Bytes of RSS per connection : 500
-----
Server open file descriptors : 10009
System-wide open files      : 28102 / 400000 (5.0% of kern.maxfiles)
kern.maxfilesperproc        : 200000
-----
Socket-buffer (mbuf) usage vs limit:
577/2483/3060 mbufs in use (current/cache/total)
0/762/762/251575 mbuf clusters in use (current/cache/total/max)
320/696/1016/125787 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37270 9k jumbo clusters in use (current/cache/total/max)
0/0/0/20964 16k jumbo clusters in use (current/cache/total/max)
14284/4920K/6353K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
-----
Tuning in force:
kern.ipc.somaxconn: 4096
kern.ipc.nmbclusters: 251575
net.inet.ip.portrange.first: 1024
net.inet.ip.portrange.last: 65535
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
=====
----- generator verdict -----
established 5000 connections (0s)
established 10000 connections (0s)
-----
SUCCESS: all 10000 connections established
holding 10000 idle connections (0s to build)
take your measurements now; Ctrl-C to release
=====
```

Figure 31: Bonus — 10,000 idle connections: all measurements plus SUCCESS: all 10000 connections established.

```

vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && sh bonus/bonus_run.sh 40000
=== bonus_run: 40000 connections on port 5000 ===
ramping...
ramp finished after 5s; letting it settle
=== done: /home/vzux/evidence/bonus/bonus_40000.txt ===
=====
BONUS MEASUREMENT  2026-09-09 18:24:49  port 5000
=====
Idle connections established : 40006
(netstat rows total 80003 = 40006 x2, both ends local)
Server PID          : 14364
=====
Server memory RSS    : 11396 KB (11.1 MB)
Server memory VSZ    : 24560 KB
Server CPU (instantaneous) : 0.0 %
Server CPU time accumulated : 0:00.19
Bytes of RSS per connection : 292
=====
Server open file descriptors : 40009
System-wide open files      : 80102 / 400000 (20.0% of kern.maxfiles)
kern.maxfilesperproc       : 200000
=====
Socket-buffer (mbuf) usage vs limit:
577/2483/3060 mbufs in use (current/cache/total)
0/762/762/251575 mbuf clusters in use (current/cache/total/max)
320/696/1016/125787 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37270 9k jumbo clusters in use (current/cache/total/max)
0/0/0/20964 16k jumbo clusters in use (current/cache/total/max)
1424K/4928K/6353K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sbufs denied
0 requests for sbufs delayed
=====
Tuning in force:
kern.ipc.somaxconn: 4096
kern.ipc.nmbclusters: 251575
net.inet.ip.portrange.first: 1024
net.inet.ip.portrange.last: 65535
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
=====
----- generator verdict -----
established 35000 connections (0s)
established 40000 connections (0s)
=====
SUCCESS: all 40000 connections established
holding 40000 idle connections (0s to build)
take your measurements now; Ctrl-C to release
=====

```

Figure 32: Bonus — 40,000 idle connections.

```

vzux@freebsd:~/A2_ROLL1_ROLL2 $ cd ~/A2_ROLL1_ROLL2 && sh bonus/bonus_run.sh 70000
=== bonus_run: 70000 connections on port 5000 ===
ramping...
ramp finished after 5s; letting it settle
=== done: /home/vzux/evidence/bonus/bonus_70000.txt ===
=====
BONUS MEASUREMENT  2026-09-09 18:25:45  port 5000
=====
Idle connections established : 70006
(netstat rows total 140003 = 70006 x2, both ends local)
Server PID          : 14437
=====
Server memory RSS    : 17532 KB (17.1 MB)
Server memory VSZ    : 28656 KB
Server CPU (instantaneous) : 0.1 %
Server CPU time accumulated : 0:00.33
Bytes of RSS per connection : 256
=====
Server open file descriptors : 70009
System-wide open files      : 140102 / 400000 (35.0% of kern.maxfiles)
kern.maxfilesperproc       : 200000
=====
Socket-buffer (mbuf) usage vs limit:
577/2483/3060 mbufs in use (current/cache/total)
0/762/762/251575 mbuf clusters in use (current/cache/total/max)
320/696/1016/125787 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37270 9k jumbo clusters in use (current/cache/total/max)
0/0/0/20964 16k jumbo clusters in use (current/cache/total/max)
1424K/4928K/6353K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sbufs denied
0 requests for sbufs delayed
=====
Tuning in force:
kern.ipc.somaxconn: 4096
kern.ipc.nmbclusters: 251575
net.inet.ip.portrange.first: 1024
net.inet.ip.portrange.last: 65535
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
=====
----- generator verdict -----
established 65000 connections (0s)
established 70000 connections (0s)
=====
SUCCESS: all 70000 connections established
holding 70000 idle connections (0s to build)
take your measurements now; Ctrl-C to release
=====

```

Figure 33: Bonus — 70,000 idle connections: 17.1 MB RSS, 0.1% CPU, 70,009 server fds, 140,102 of 400,000 system-wide files, zero mbuf clusters.

```

vzux@freebsd:~/A2_ROLL1_ROLL2 $ vmstat -z | head -1; vmstat -z | grep "socket:"; sysctl kern.ipc.maxsockets kern.maxfiles
ITEM      SIZE  LIMIT  USED   FREE   REQ  FAIL  SLEEP  XOOM
socket:    1024, 400000, 664, 139728, 1084704, 110, 0, 0
kern.ipc.maxsockets: 400000
kern.maxfiles: 400000
vzux@freebsd:~/A2_ROLL1_ROLL2 $

```

Figure 34: Bonus — the bottleneck evidence. The socket UMA zone now at LIMIT 400000, with FAIL 110 still recorded from when the limit was 129,967.